Intro to Cryptography

CS 161 Spring 2026 - Lecture 6



Hello, I am Henry.

- I'm teaching the cryptography and network-security units
- I always go by "Henry" – please feel free to call me that!

Questions in lecture

- Feel free to ask via Zoom chat; I will try to monitor it during lecture

Announcements

- Project 1 Checkpoint is due **this** Friday (2/6)
  - Project 1 Party will be held **5-7PM Thursday (2/5) in Wozniak Lounge (Soda Hall)**
- Homework 2 is released and will be due **next** Friday (2/13)

Today

- What is cryptography?
- Secret-key encryption
- First security definition: Semantic security for one-time key
- One-time pad
- Second security defn: Semantic security for many-time key (CPA Security)

Why study cryptography?

- It is built on beautiful theory.
- It works in practice.
- It solves an important everyday problem.

(I first heard this from Dan Spielman, as “things to want from an algorithm.”)

Draws on algorithms, complexity, architecture, number theory, …

Cryptography is a “success story” within computer security

- Computer systems rarely fail because the cryptography failed (cryptocurrency software may be the exception)
- Q: What are the common causes of security failure?

Cryptography: Securing **communication** and **computation** against adversarial behavior.



Alice

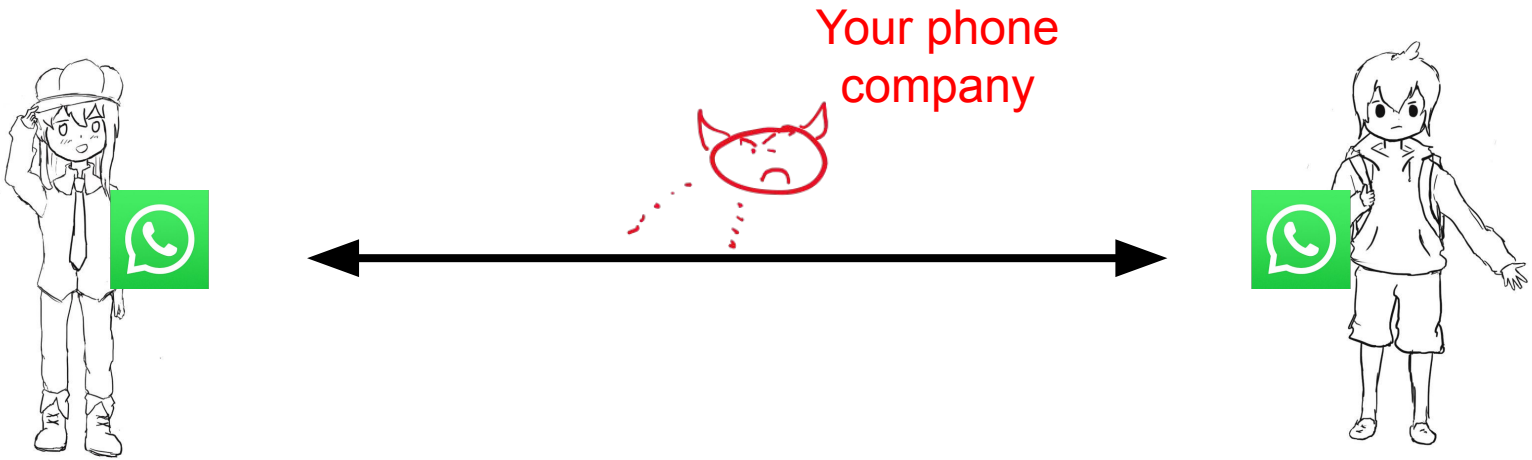




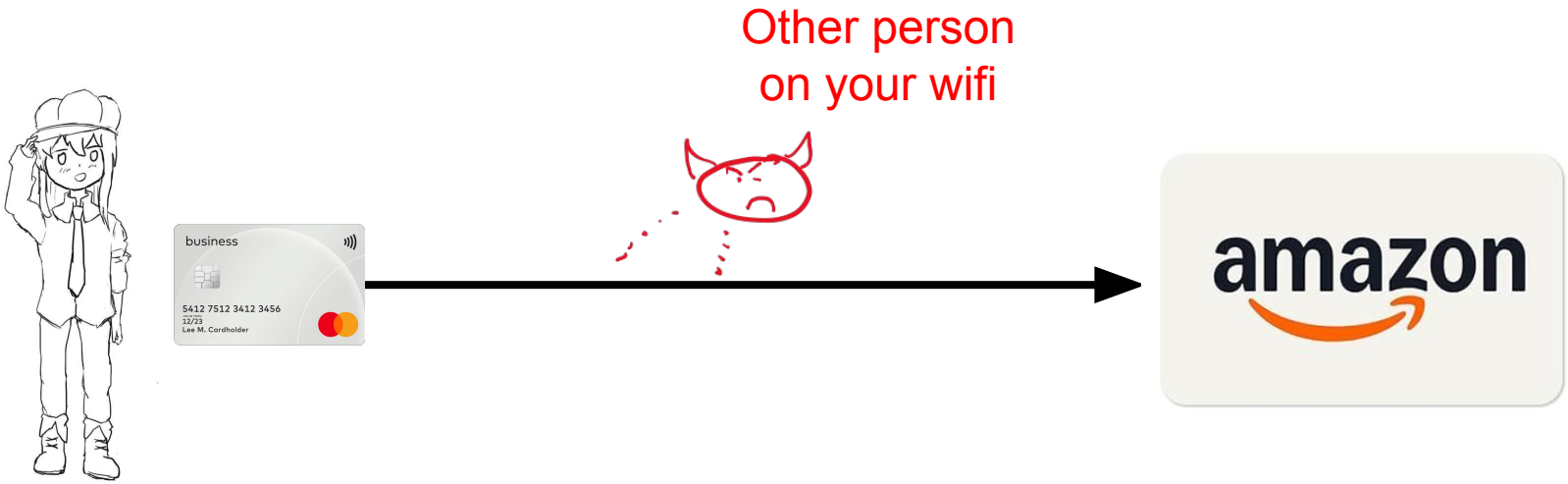
Bob

Goal: Prevent attacker from
- tampering with messages (“Authenticity”)
- reading messages (“Confidentiality”)

Example: Encrypted messaging



Example: Online purchases



Example: ATMs



Criminal group



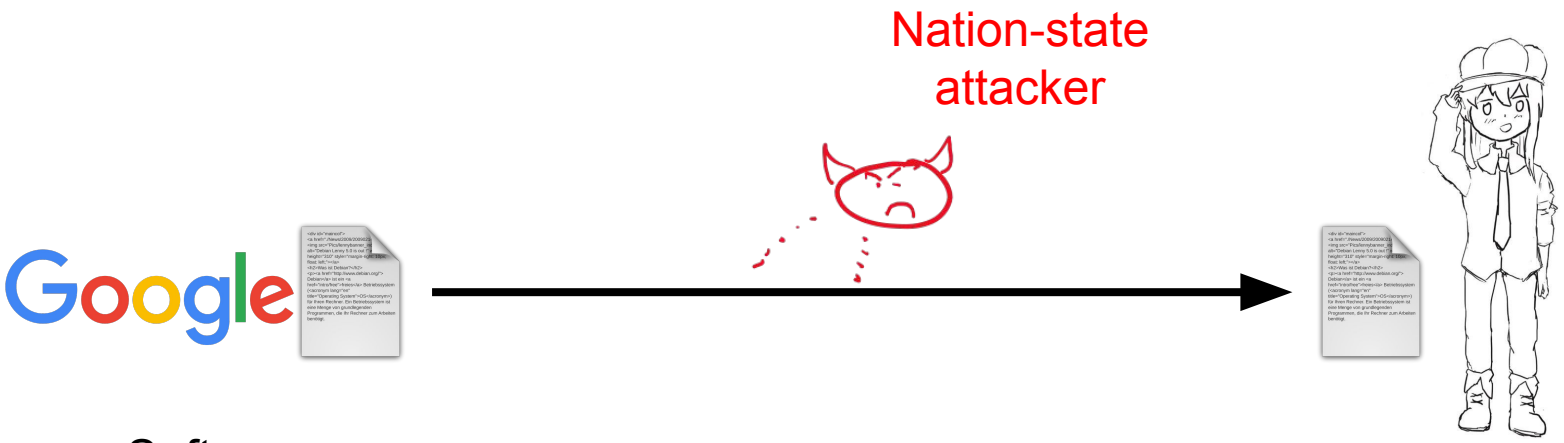




ATM

Bank

Example: Software updates



The cryptography landscape

1. **What setting are we in?**
   - Alice and Bob share a secret key → Secret-key crypto (or “Symmetric key”)
     - Key = random bit-string unknown to the adversary
   - Alice and Bob have no shared secrets (?!?!?) → Public-key crypto (or “Asymmetric key”)

2. **What are we trying to protect?**
   - Confidentiality → Encryption
     - An attacker in the middle “cannot learn information” about Alice’s message
   - Authenticity/Integrity → Message auth codes. or digital signatures
     - Bob gets the message that Alice sent (or knows that he didn’t)
     - Often, we want authenticity without confidentiality (When?)

What you will learn in the next three weeks

	Secret-key crypto	Public-key crypto
Encryption		
Authentication		

What you will learn in the next three weeks

	Secret-key crypto	**Public-key crypto**
Encryption	– One-time pad – Counter-mode encryption – CBC encryption	– Key exchange (e.g., Diffie-Hellman) – Public-key encryption (e.g., ElGamal)
Authentication	– Message auth. codes (“MACs”)	– Digital signatures (e.g. RSA sigs)

Building blocks	– Pseudorandom functions – Block ciphers (e.g., AES) – Hash functions (e.g., SHA2)	– Discrete-log problem



Fasten your seatbelts…



Photo credit: NASA

This class

- Will make you a more informed user of crypto tools
  - Which tool to reach for
  - How to use it
  - Common implementation issues

- This course will **NOT** teach you how to design crypto tools
  - That is a whole other can of worms

If you remember nothing else about cryptography…

DO: Use standard cryptosystems. e.g., AES, SHA2

DON'T: Use a homemade cryptosystem.

→ Security of crypto rests on unproven assumptions e.g., factoring is hard

→ Confidence comes from years of cryptanalysis

DO: Use standard cryptography libraries, in a safe language.

DON'T: Implement your own cryptography library.

→ Crypto code is exceptionally subtle (hard to test, etc.)

→ Pre-existing libraries are often more faster too

DO: Use standard protocols. e.g., TLS, Signal

DON'T: Implement your own protocol. :-(

Secret-Key Encryption



Textbook Chapter 6.1

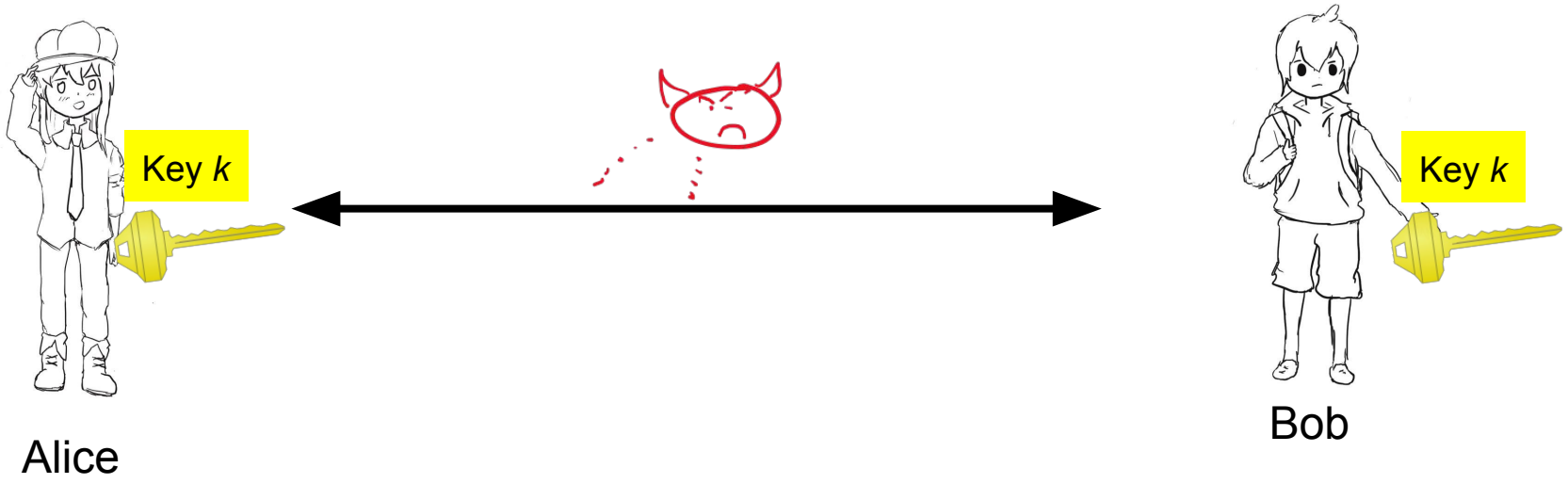
Cryptography roadmap

	Secret-key crypto	Public-key crypto
Encryption		
Authentication		



Symmetric-key encryption

- Recall: Alice and Bob share a secret string (e.g., 128 bits), unknown to the adversary
  - Can exchange secret in person during a setup phase
  - Or, can use public-key crypto (in a few lectures)



Kerckhoff's Principle

Principle: A cryptosystem should be secure even if an attacker knows everything but the secret keys.
→ Secret keys are the only secrets

- We will always assume: The attacker knows the algorithm.
  - The secret keys are the only things hidden from the attacker.
- Why is this a good principle?
  - It's easier to keep a small (e.g., 128-bit) key secret than a big algorithm
  - If your keys get leaked, it's easier to change keys than to change algorithms
  - Can crowdsource the design of the algorithm
    - The most widely used encryption scheme today (AES) was designed via a public competition

Secret-Key Encryption: Syntax

A secret-key encryption scheme consists of three algorithms:

- KeyGen() → *K*: Generate a key *K*
- Enc(*K*, *M*) → *C*: Encrypt a **plaintext/message** *M* using the key *K* to produce **ciphertext** *C*
- Dec(*K*, *C*) → *M*: Decrypt a ciphertext *C* using the key *K*

Message space = Set of all messages that a particular scheme can encrypt

E.g., All 1024-byte strings, or all 16-digit credit card numbers, or all bitstrings.

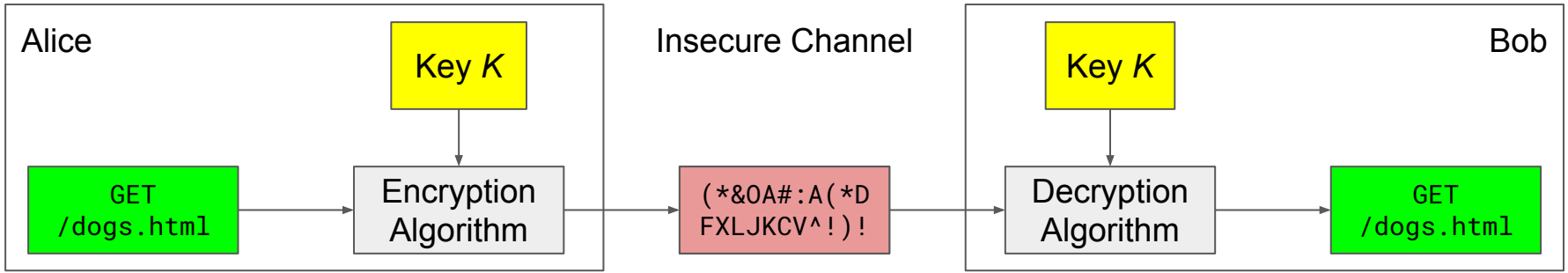
Secret-Key Encryption: Syntax

A secret-key encryption scheme consists of three algorithms:

- KeyGen() → *K*: Generate a key *K*
- Enc(*K*, *M*) → *C*: Encrypt a **plaintext/message** *M* using the key *K* to produce **ciphertext** *C*
- Dec(*K*, *C*) → *M*: Decrypt a ciphertext *C* using the key *K*

Message space = Set of all messages that a particular scheme can encrypt

E.g., All 1024-byte strings, or all 16-digit credit card numbers, or all bitstrings.



Symmetric-Key Encryption: Desired properties

Correctness: If you encrypt and decrypt with the same key,
you should get the original message back.

For all $K \leftarrow$ KeyGen() and all M in msg space: Dec(K, Enc(K, M)) = M

Efficiency: Encryption/decryption algorithms should be fast.

– In theory: Polynomial time

– In practice: "Fast enough" = GBs/second on your phone/laptop

Security: ???

How should we define security for encryption schemes?

- This is a very tricky question
  - Humanity used encryption schemes for ≥ 2500 years before finding a satisfying security definition!
- There are many right answers: there are many definitions of security
  - Definition encodes **attacker's power** and **attacker's goal**
  - Which attack matters depends on the application

- Attempt 1: "Given the ciphertext, the attacker should not be able to recover the entire message."



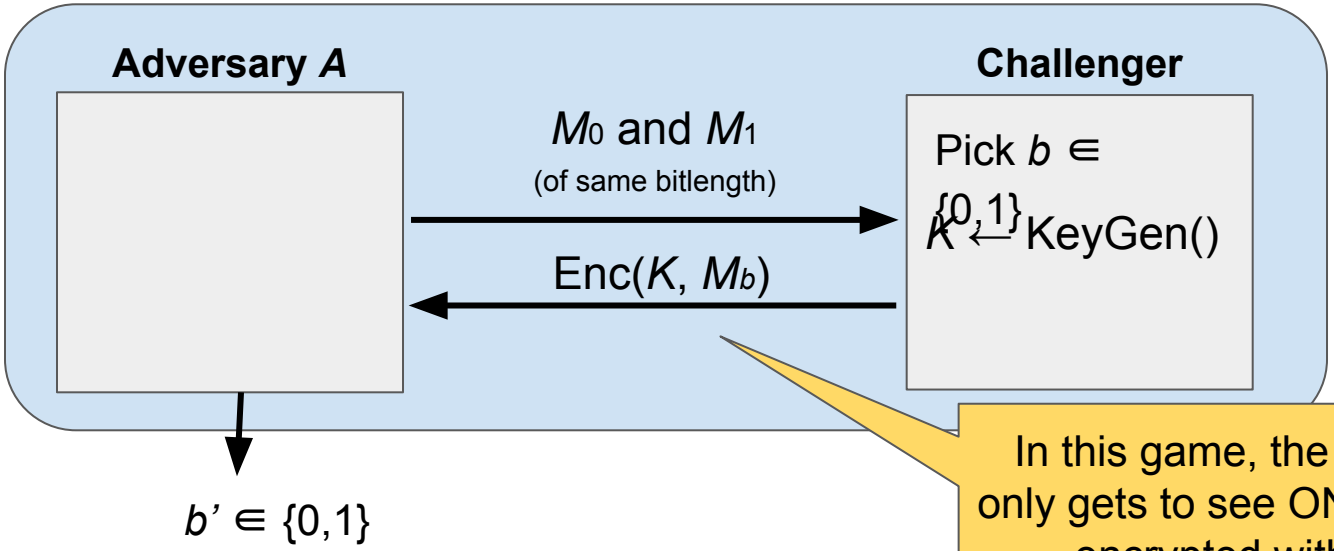
Our first definition:

Semantic security for one-time key

Idea: "Given the ciphertext, the attacker learn *no new information* about the plaintext, (apart from the plaintext length)."

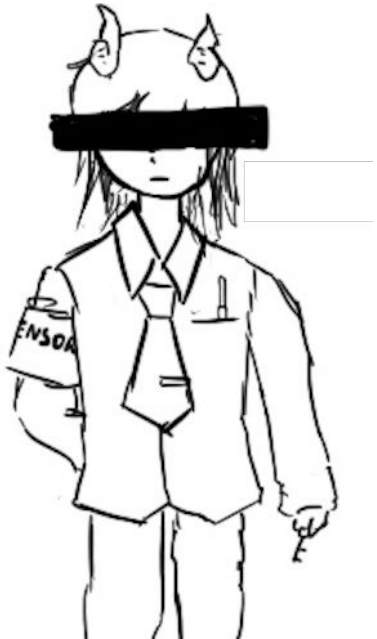
- We make this precise with an experiment/security **game**
  - Between the adversary (Eve) and a challenger (playing the role of Alice/honest parties)
- The game:
  - Adversary picks two messages M_0 and M_1 of equal length
  - Challenger encrypts M_b for random b in {0,1}
  - Adversary "wins" if it guesses b correctly.
  - We say that enc. scheme is secure if adversary basically can't beat guessing at random
- Intuition
  - If ciphertext leaks **no information** about plaintext, adversary can only guess with probability ½
  - Enc. scheme is secure if adversary can't do much better than that

Definition: Semantic security for one-time key



An encryption scheme has *semantic security for a one-time key* if, for all "efficient" adversaries *A*, | Pr[*A* outputs *b*] - ½ | ≤ "negligible."

One-Time Pads



Textbook Chapter 6.2 & 6.3

One-time pad

Our first encryption scheme

Why study it:

- It is a classic cryptosystem, used since 1880s
- It is very elegant—as simple as it gets
- It is “perfectly secure”
  - Secure against an attacker running in $2^{100000000000}$ time

Review: XOR

The XOR operator takes two bits and outputs one bit:

$0 \oplus 0 = 0$
$0 \oplus 1 = 1$
$1 \oplus 0 = 1$
$1 \oplus 1 = 0$

Useful properties of XOR:

$x \oplus 0 = x$
$x \oplus x = 0$
$x \oplus y = y \oplus x$
$(x \oplus y) \oplus z = x \oplus (y \oplus z)$
$(x \oplus y) \oplus x = y$

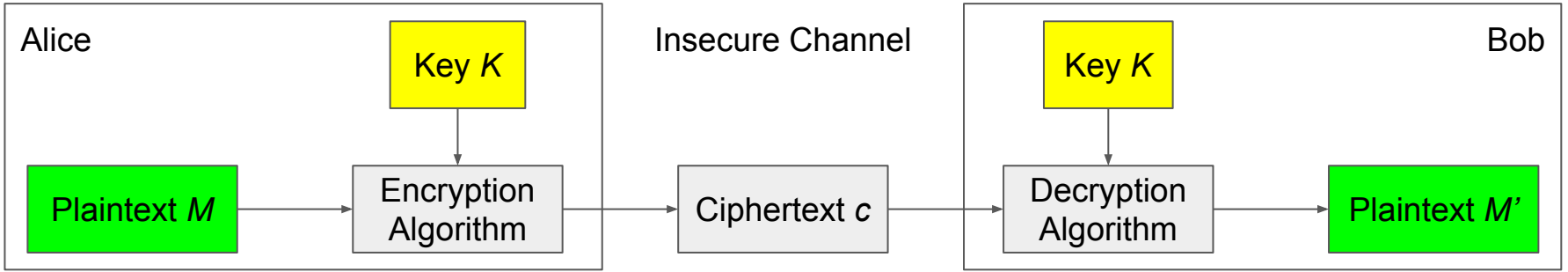
Review: XOR Algebra

- Algebra works on XOR too

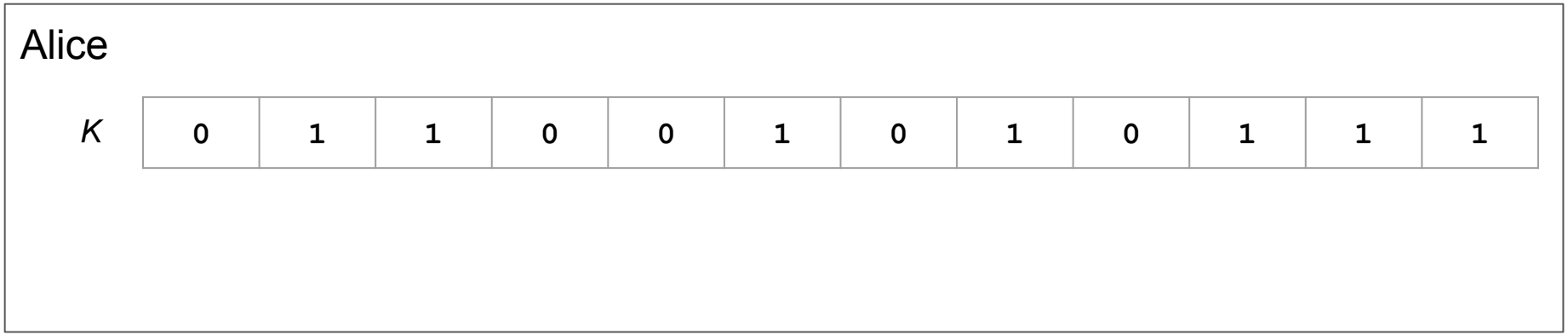
$y \oplus 1 = 0$	Goal: Solve for y
$y \oplus 1 \oplus 1 = 0 \oplus 1$	XOR both sides by 1
$y = 1$	Simplify with identities

- XOR is just addition modulo 2

Reminder: Secret-Key Encryption



One-Time Pads: Key Generation



- The key *K* is a random *n*-bit string, for some parameter *n*
  - We are in the secret-key setting: Alice and Bob both know *K*

- The message space is the set of *n*-bit strings
  - i.e., we can encrypt *n*-bit messages
  - Can always represent your text as a bitstring, e.g., using ASCII

One-Time Pads: Encryption

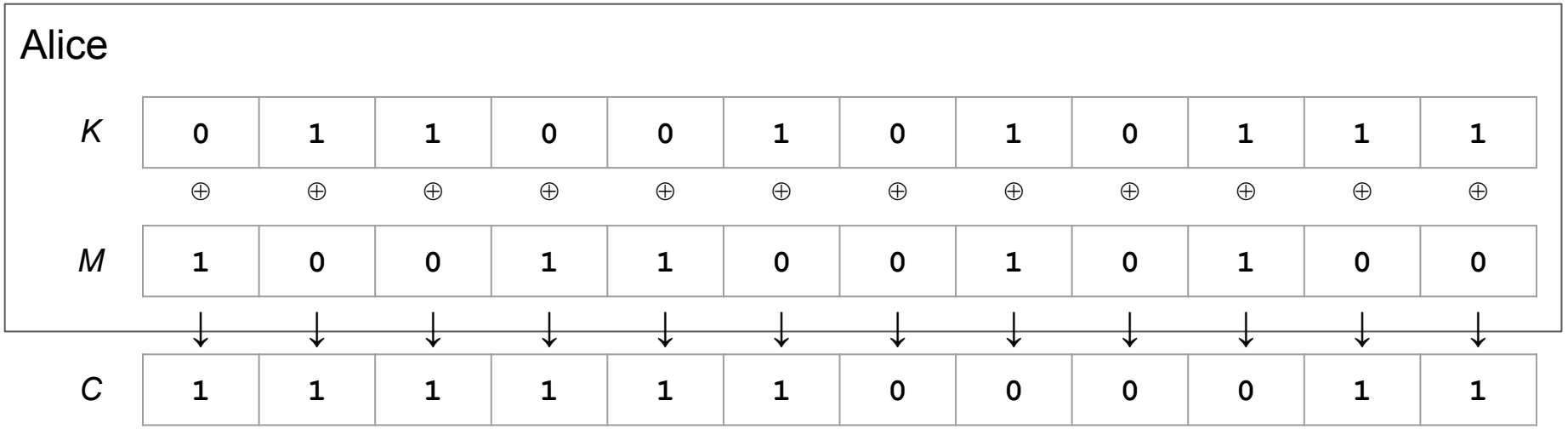
Alice

K	0	1	1	0	0	1	0	1	0	1	1	1
M	1	0	0	1	1	0	0	1	0	1	0	0

The plaintext *M* is the bitstring that Alice wants to encrypt.

Idea: Use XOR to scramble up *M* with the bits of *K*.

One-Time Pads: Encryption



Encryption algorithm: XOR each bit of *K* with the matching bit in *M*.

The ciphertext *C* is the encrypted bitstring that Alice sends to Bob over the insecure channel.

One-Time Pads: Decryption

Bob

K	0	1	1	0	0	1	0	1	0	1	1	1
C	1	1	1	1	1	1	0	0	0	0	1	1

Bob receives the ciphertext *C*. Bob knows the key *K*. How does Bob recover *M*?

One-Time Pads: Decryption

Bob

K	0	1	1	0	0	1	0	1	0	1	1	1
	⊕	⊕	⊕	⊕	⊕	⊕	⊕	⊕	⊕	⊕	⊕	⊕
C	1	1	1	1	1	1	0	0	0	0	1	1
	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
M	1	0	0	1	1	0	0	1	0	1	0	0

Decryption algorithm: XOR each bit of *K* with the matching bit in *C*.

One-Time Pad

- KeyGen()
  - Choose a random *n*-bit string, for some parameter *n*

- Enc(*K*, *M*) = $K \oplus M$ (As simple as it gets!)
  - Bitwise XOR *M* and *K* to produce *C*
    - In other words: XOR the *i*th bit of the plaintext with the *i*th bit of the key.
    - $C_i = K_i \oplus M_i$

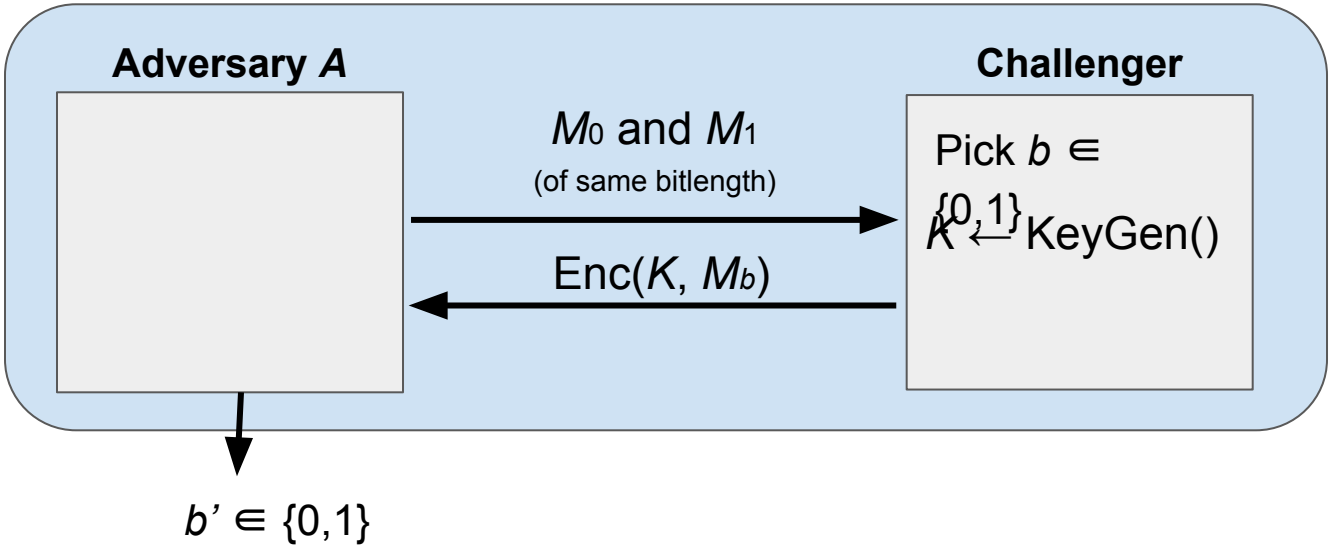
- Dec(*K*, *C*) = $K \oplus C$
  - Bitwise XOR *C* and *K* to produce *M*
    - $M_i = K_i \oplus C_i$

One-Time Pad: Correctness

- Correctness: If we encrypt and then decrypt, we should get the original message back

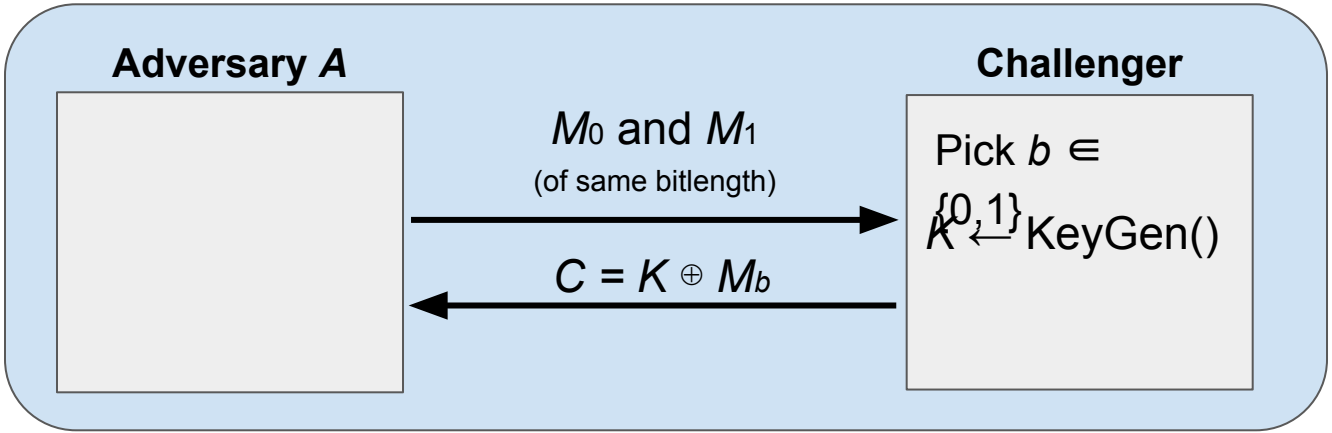
$$
\begin{aligned}
\text{Enc}(K, M) &= K \oplus M && \text{Definition of encryption} \\
\text{Dec}(K, \text{Enc}(K, M)) &= \text{Dec}(K, K \oplus M) && \text{Decrypting the ciphertext} \\
&= K \oplus (K \oplus M) && \text{Definition of decryption} \\
&= M && \text{XOR property}
\end{aligned}
$$

Recall our security definition: Semantic security for one-time key



An encryption scheme has *semantic security for a one-time key* if, for all "efficient" adversaries A, $|\Pr[A \text{ outputs } b] - \frac{1}{2}| \leq$ "negligible."

Claim: One-Time Pad is semantically secure for a one-time key



Possibility 0: Alice sends Enc(K, M_0)

The ciphertext is $C = K \oplus M_0$

Therefore, $K = C \oplus M_0$

Possibility 1: Alice sends Enc(K, M_1)

The ciphertext is $C = K \oplus M_1$

Therefore, $K = C \oplus M_1$

Claim: One-Time Pad is semantically secure for a one-time key

Possibility 0: Alice sends Enc(K, M_0)

The ciphertext is $C = K \oplus M_0$

Therefore, $K = C \oplus M_0$

Possibility 1: Alice sends Enc(K, M_1)

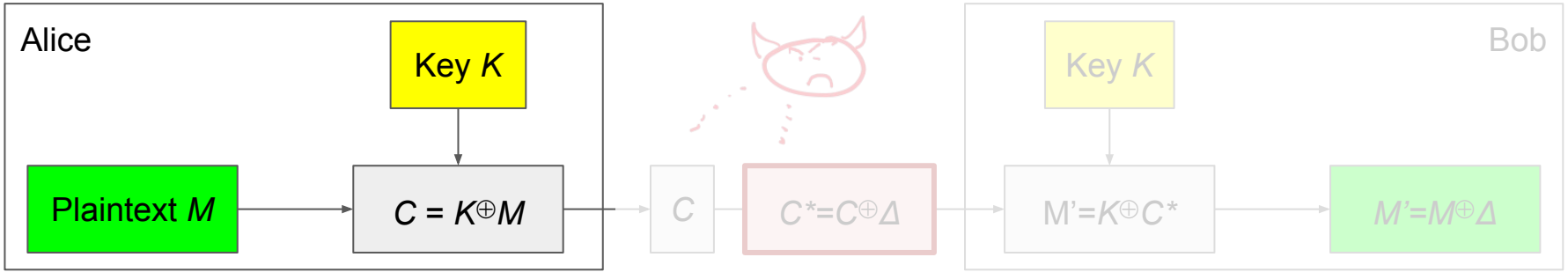
The ciphertext is $C = K \oplus M_1$

Therefore, $K = C \oplus M_1$

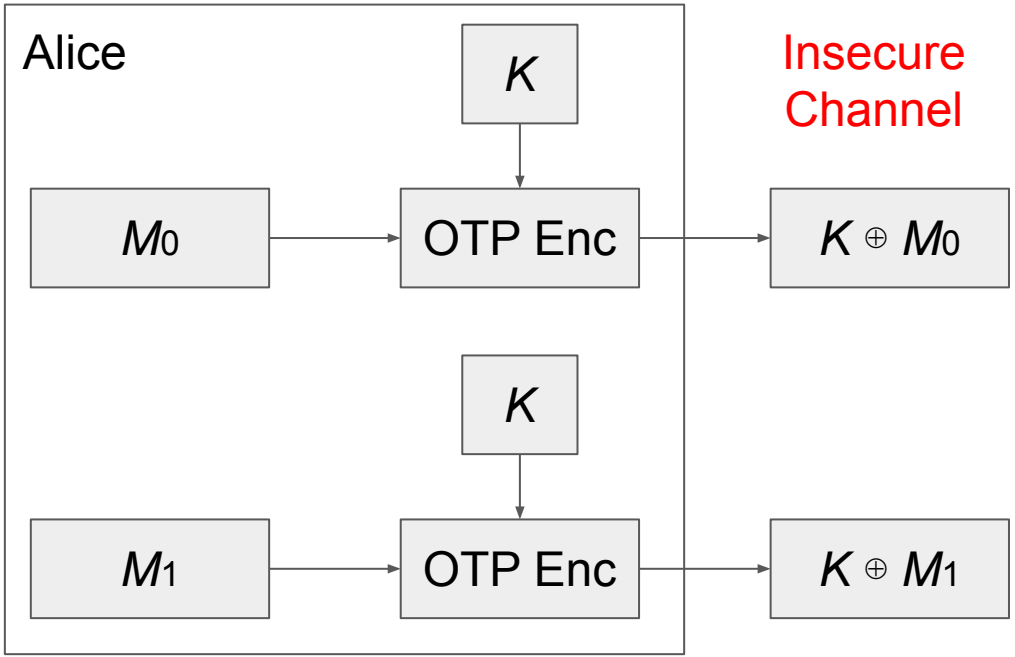


- K is a random string ⇒ Both possibilities are equally likely!
- Attacker learns no information about plaintext (except its length)
- Probability that attacker wins = ½ = chance of guessing random
  - Scheme is “perfectly secure” or “information-theoretically secure”
  - Secure even if attacker has unlimited computation time

Does one-time pad protect against tampering?



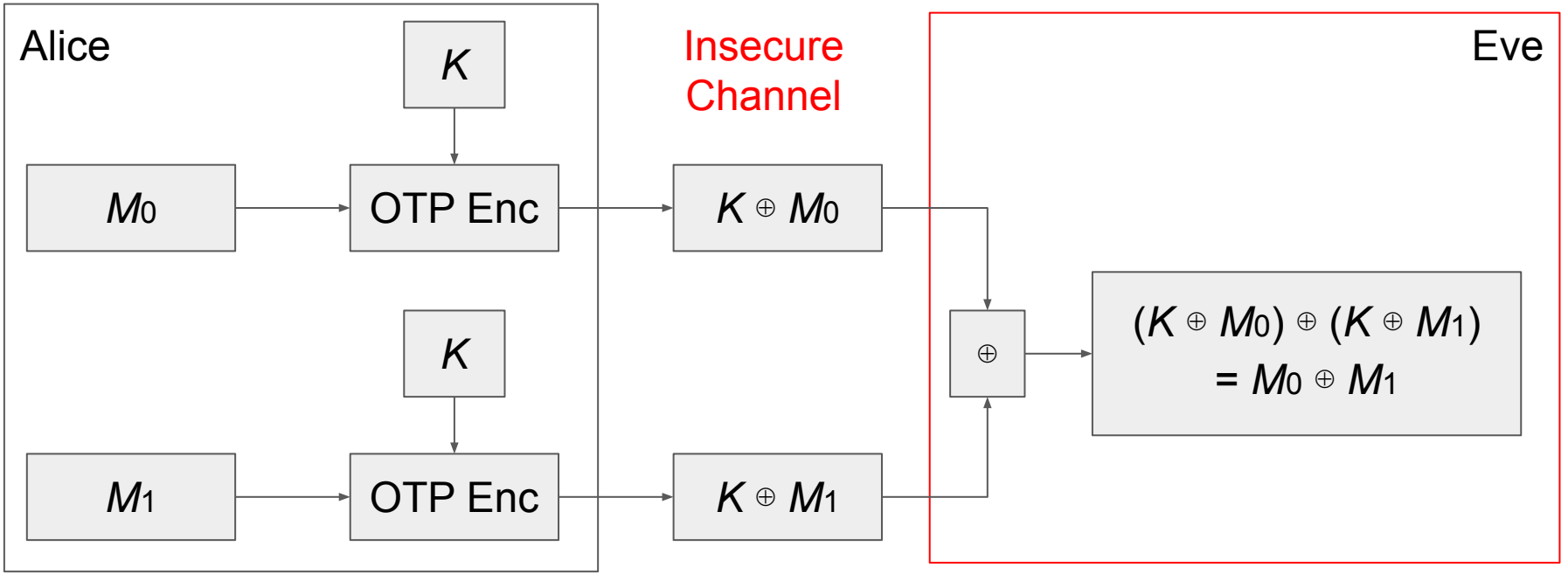
Two-Time Pads?



Eve sees two ciphertexts over the insecure channel.

What if we use the same key *K* to encrypt two different messages?

Two-Time Pads?



If Eve XORs the two ciphertexts, she learns $M_0 \oplus M_1$!

Two-Time Pads?

- What if we use the same key *twice*?
  - Alice encrypts M_0 and M_1 with the same key
  - Eve observes $K \oplus M_0$ and $K \oplus M_1$
  - Eve computes $(K \oplus M_0) \oplus (K \oplus M_1) = M_0 \oplus M_1$
    - Recall the XOR property: the K's cancel out
- Eve has learned $M_0 \oplus M_1$. This is partial information about the messages!
  - In other words, Eve knows which bits in M_0 match bits in M_1
  - If Eve knows M_0, she can deduce M_1 (and vice-versa)
- What went wrong?
  - **Problem:** Our security definition was not very strong
  - Semantic security for one-time key doesn't guarantee anything if you use the key twice!

→ Alice and Bob must use a fresh one-time key for every message! :-(

Two-Time Pads in Practice: VENONA

- Soviet spies used one-time pads for communication from their spies in the US
- During WWII, the Soviets started reusing key material
  - Uncertain whether it was just the cost of generating pads or what…
- VENONA was a US cryptanalysis project designed to break these messages
  - Included confirming/identifying the spies targeting the US Manhattan project
  - Project continued until 1980!
- Not declassified until 1995!
  - So secret even President Truman wasn't informed about it
  - The Soviets found out about it in 1949 through their spy Ken Philby, but their one-time pad reuse was fixed after 1948 anyway
- **Takeaway**: Otherwise-secure cryptographic systems can fail very badly if used improperly!

One-time pad: Summary

Pros:

- Simple
- Easy to implement
- Perfectly secure

Cons:

- Key is as long as message
- Does not protect against tampering attacks
- Can only encrypt one message with each key

Stronger security definition: IND-CPA Security

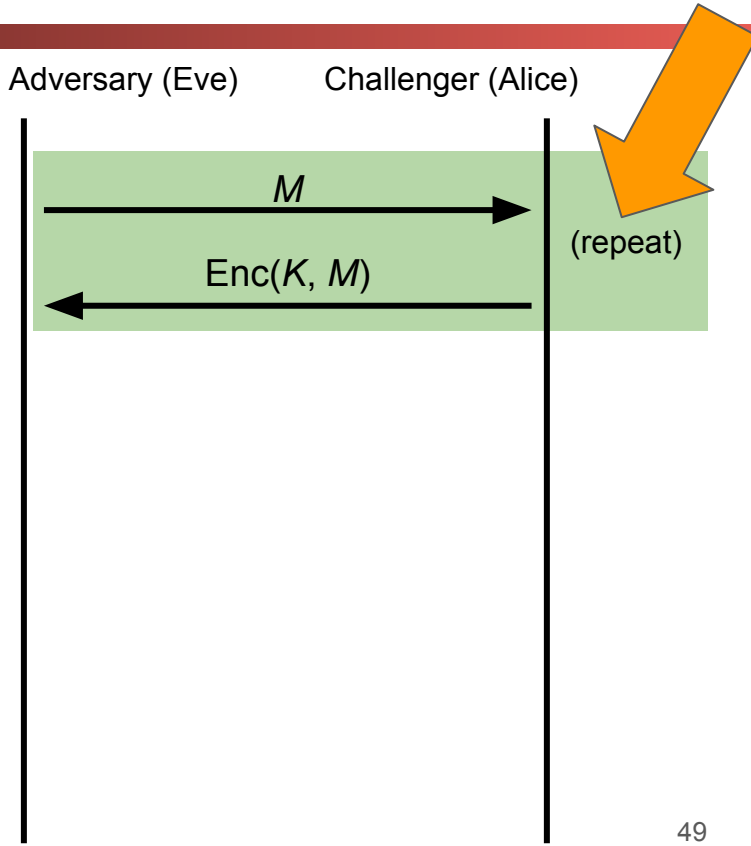
Textbook Chapter 5.3–5.9

Security with key reuse: IND-CPA

- Let's extend our earlier security definition to handle key reuse
  - Adversary can see encryption of many messages of its choice
  - Even then, adversary shouldn't be able to guess which message Alice sent with probability much better than 1/2.
- This definition is called "**semantic security**" or **IND-CPA** or **CPA** security (indistinguishability under chosen plaintext attack)
  - Requires enc scheme to leak nothing even under key reuse
  - The entry-level security definition for encryption schemes
- In deployed systems, we ask for MORE than IND-CPA security—why?

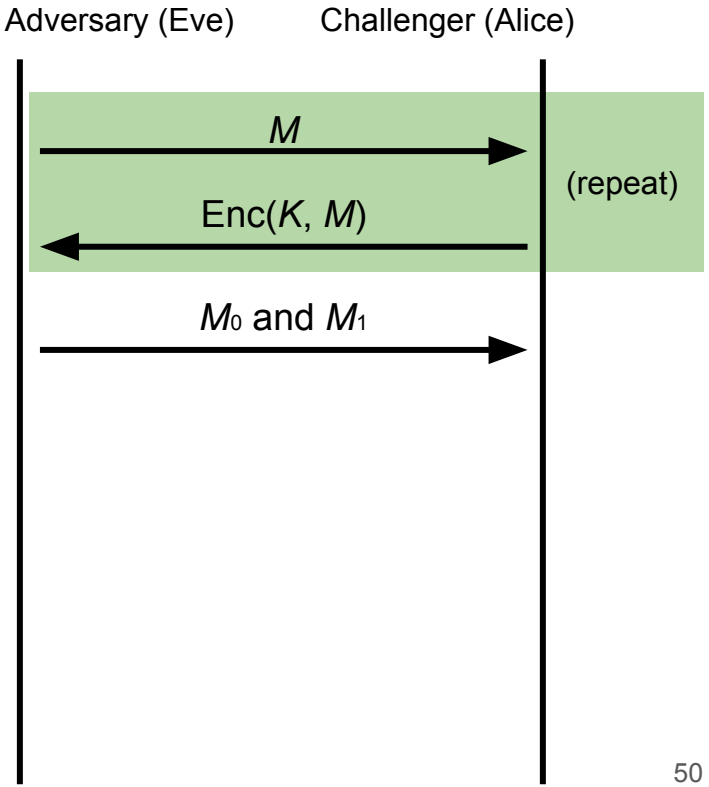
Security with key reuse: IND-CPA

1. Adversary may ask for encryption**s** of plaintext**s** of its choice



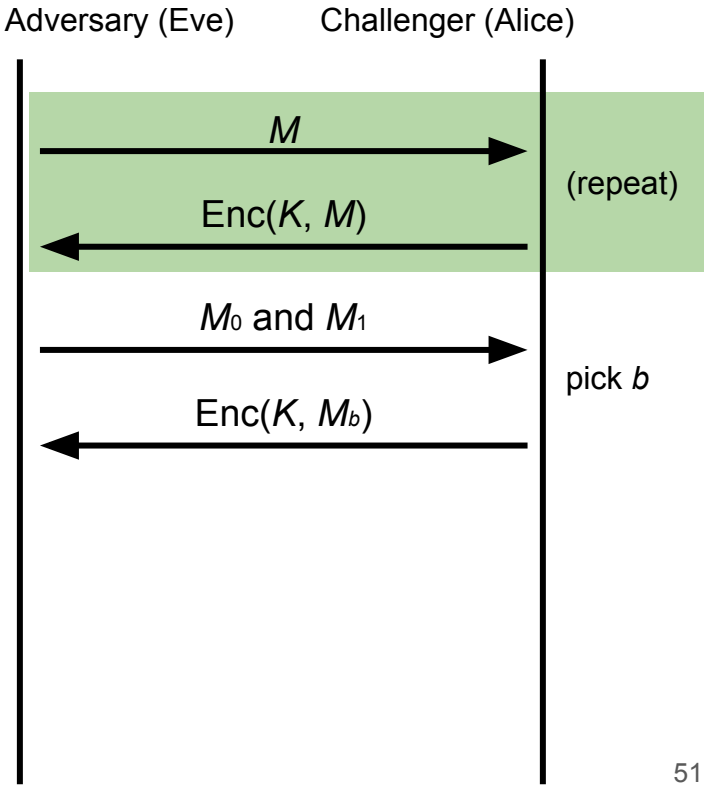
Security with key reuse: IND-CPA

1. Adversary may ask for encryptions of plaintexts of its choice
2. Adversary sends plaintexts M_0 and M_1 to challenger (of equal length)



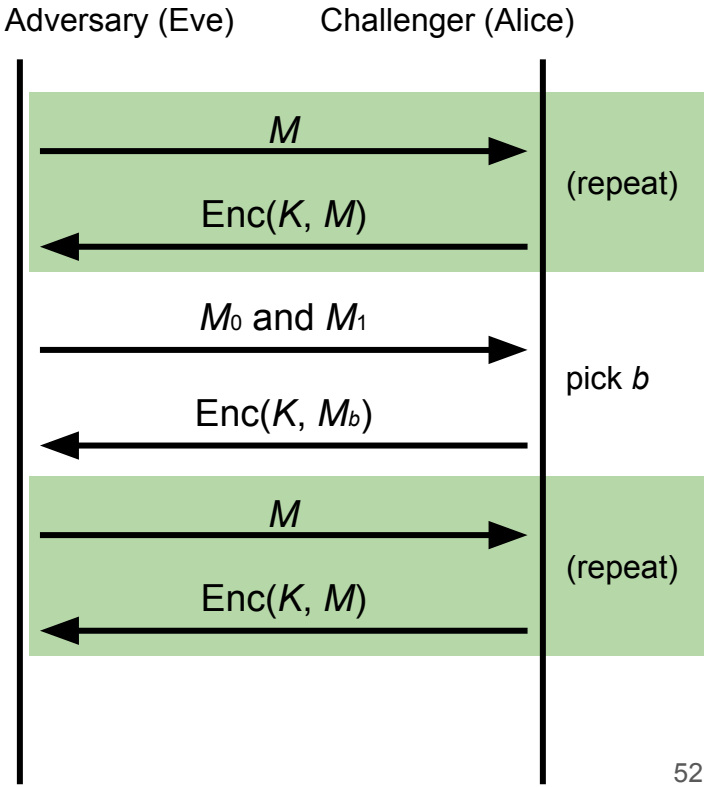
Security with key reuse: IND-CPA

1. Adversary may ask for encryptions of plaintexts of its choice
2. Adversary sends plaintexts M_0 and M_1 to challenger (of equal length)
3. Challenger randomly chooses either M_0 or M_1 to encrypt and sends the ciphertext back
   a. Challenger does not tell adversary which one was encrypted!



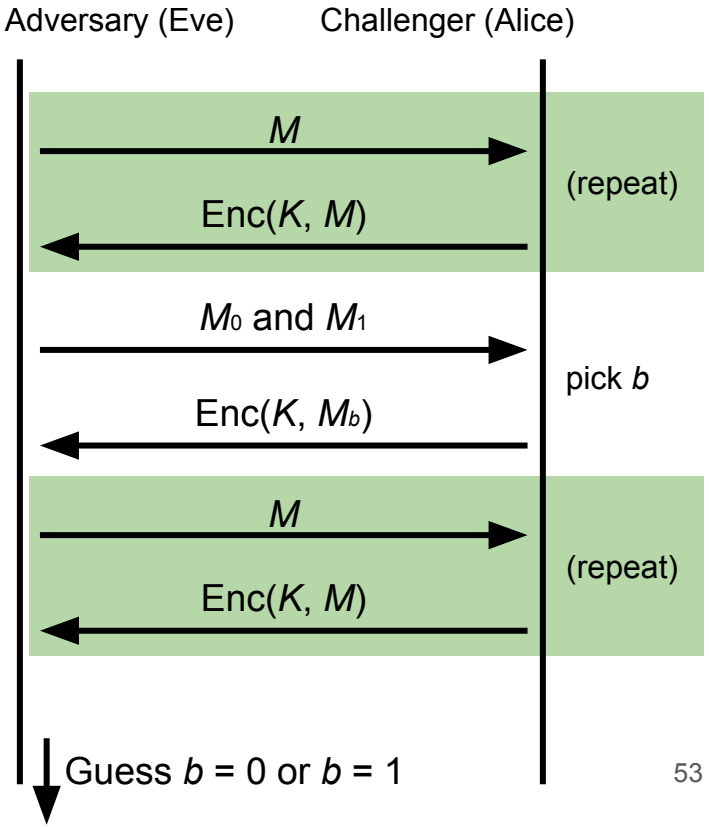
Security with key reuse: IND-CPA

1. Adversary may ask for encryptions of plaintexts of its choice
2. Adversary sends plaintexts M_0 and M_1 to challenger (of equal length)
3. Challenger randomly chooses either M_0 or M_1 to encrypt and sends the ciphertext back
   a. Challenger does not tell adversary which one was encrypted!
4. Adversary may ask for more encryptions



Security with key reuse: IND-CPA

1. Adversary may ask for encryptions of plaintexts of its choice
2. Adversary sends plaintexts M_0 and M_1 to challenger (of equal length)
3. Challenger randomly chooses either M_0 or M_1 to encrypt and sends the ciphertext back
   a. Challenger does not tell adversary which one was encrypted!
4. Adversary may ask for more encryptions
5. Adversary guesses whether challenger encrypted M_0 or M_1



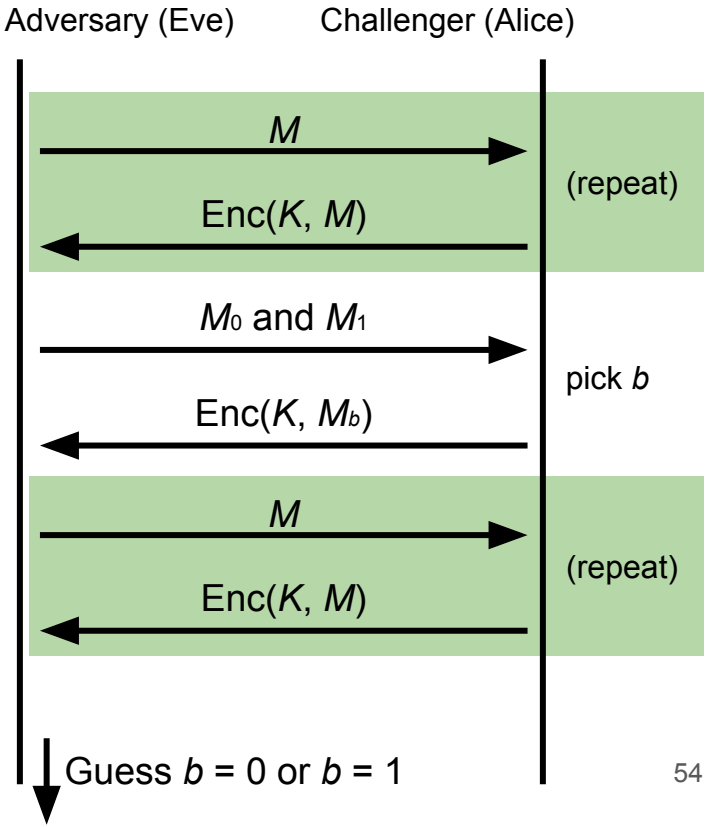
Security with key reuse: IND-CPA

An encryption scheme is *IND-CPA secure* if, for all "efficient" adversaries *A*,

$$|\ \Pr[A \text{ outputs } b] - \tfrac{1}{2}\ | \leq \text{"negligible."}$$

"Efficient" = less than 2^{128} time (in practice)
= polynomial time (in theory)

"Negligible" = less than 2^{-128} (in practice)
= out of scope (in theory)



Security with key reuse: IND-CPA

- How does adversary guess whether M_0 or M_1 was encrypted?
  What strategy does she use?
  - We don't *assume* she uses a particular strategy; adversary could use any possible strategy

- **To prove insecurity:** Must show that there exists at least *one* strategy that wins the IND-CPA game with probability >> 1/2
  - 1/2 is the probability of winning by random guessing
  - If you can do better than random, then the ciphertext has leaked information

- **To prove security:** Must show that for ***all*** efficient attackers/Eve-s, the probability of winning the IND-CPA game is at most 1/2

On “efficient” and “negligible”

IND-CPA security says: “No **efficient** adversary can win the IND-CPA game with probability better than ½ + **negligible**.”

I said that often in practice:

- “efficient” adversary is one that runs in time << 2^{128}
- “negligible” probability is ~ 2^{-80}

Where do these magic numbers come from? Empirical estimates…

Summary

- One-time pad
  - A simple secret-key encryption scheme
  - Many limitations: Large, one-time-use key
- Security definitions
  - Phrased using a game between challenger and adversary
  - IND-CPA security: Security even if use same key to encrypt many msgs

Next time:

- Counter mode: A CPA-secure encryption scheme
- Pseudorandom functions

What we learned

	Secret-key crypto	**Public-key crypto**
Encryption	– One-time pad	
Authentication		

The end